

École Nationale des Sciences Appliquées, Tanger
Filière Génie Informatique
Année Universitaire:
2025-2026

Mini Projet Spring Boot & Thymeleaf

Réalisé par :

AZZMAN Nour Elimane

SOMMAIRE

Introduction

- Présentation du projet
- Objectifs pédagogiques et techniques

Chapitre 1 : Modèle Conceptuel des Données (MCD)

- Présentation générale du MCD
- Description des entités (Élève, Filière, Cours, Dossier Administratif)
- Relations entre les entités
- Justification du modèle choisi

Chapitre 2 : Architecture de l'Application

- Présentation générale de l'architecture
- Architecture MVC (Model – View – Controller)
- Avantages de l'architecture MVC dans ce projet

Chapitre 3 : Gestion des dépendances – Fichier pom.xml

Chapitre 4 : Persistance des données avec JPA et MySQL

- Présentation de JPA et de l'ORM
- Configuration de la base de données MySQL
- Mapping des entités avec JPA
- Gestion des relations entre les entités

Chapitre 5 : Implémentation des opérations CRUD

- CRUD de l'entité Élève
- CRUD de l'entité Filière
- CRUD de l'entité Cours
- Gestion du Dossier Administratif
- Règles métiers appliquées lors des opérations

Chapitre 6 : Implémentation des vues avec Thymeleaf

- Présentation de Thymeleaf
- Création des pages HTML dynamiques
- Interaction entre contrôleurs et vues
- Utilisation de Bootstrap pour l'interface utilisateur

Conclusion

Introduction

Ce projet consiste à développer une application Web de gestion académique en utilisant le framework **Spring Boot**, avec **Spring Data JPA** pour la gestion des données et **Thymeleaf** pour la création des interfaces graphiques. L'application permet de gérer les entités principales d'un établissement scolaire, notamment les élèves, les filières, les cours et les dossiers administratifs, tout en respectant les règles de gestion définies.

L'objectif de ce travail est à la fois pédagogique et technique : il vise à appliquer les concepts étudiés en cours, tels que l'architecture MVC, la modélisation des données, les relations entre entités et les opérations CRUD, tout en développant une application fonctionnelle et bien structurée. Ce projet permet ainsi de renforcer la compréhension du développement d'applications Web avec Spring Boot et d'acquérir une première expérience concrète dans la conception d'un système de gestion académique.

Chapitre 1 : Modèle Conceptuel des Données (MCD)

Présentation générale du Modèle Conceptuel des Données

Le Modèle Conceptuel des Données (MCD) permet de représenter de manière claire et structurée les données manipulées par l'application ainsi que les relations qui existent entre elles. Dans ce projet de gestion académique, le MCD a été conçu afin de répondre aux besoins fonctionnels du système scolaire, en assurant une organisation cohérente des informations relatives aux élèves, aux filières, aux cours et aux dossiers administratifs.

Description des entités

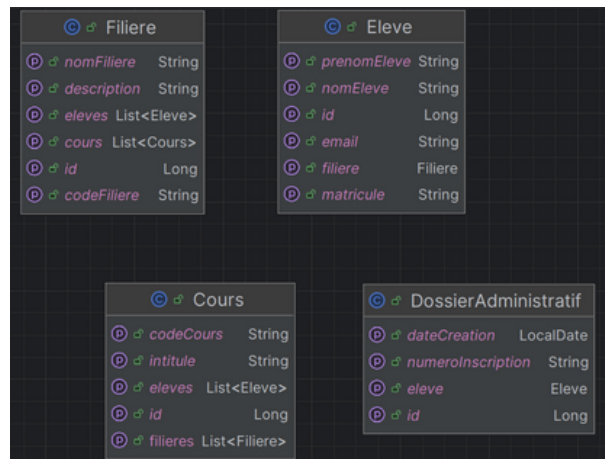
Le MCD du projet repose sur quatre entités principales :

- L'entité **Élève** représente un étudiant inscrit dans l'établissement. Elle contient des informations personnelles telles que le matricule, le nom, le prénom et l'email. Chaque élève est rattaché à une seule filière et possède un dossier administratif unique.
- L'entité **Filière** représente une spécialité ou un parcours académique. Elle est définie par un code, un nom et une description. Une filière peut regrouper plusieurs élèves et proposer plusieurs cours.
- L'entité **Cours** correspond aux enseignements dispensés dans l'établissement. Chaque cours est identifié par un code et un intitulé. Un cours peut être associé à plusieurs filières et suivi par plusieurs élèves.
- L'entité **DossierAdministratif** permet de gérer les informations administratives liées à un élève. Elle contient le numéro d'inscription et la date de création du dossier. Chaque dossier administratif est strictement associé à un seul élève.

Ces entités reflètent fidèlement la structure du domaine académique ciblé par l'application.

Chapitre 1 : Modèle Conceptuel des Données (MCD)

Description des entités

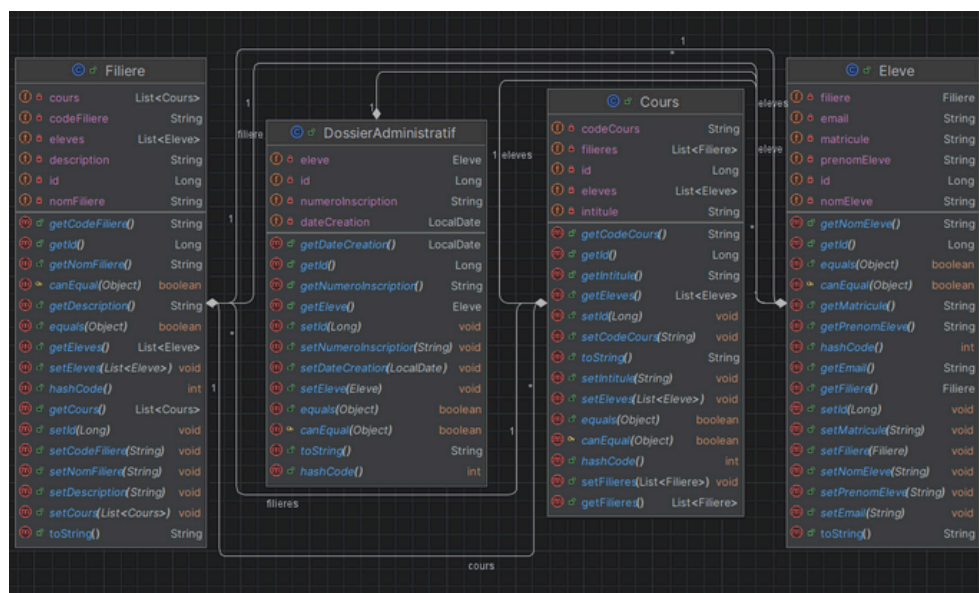


Relations entre les entités

Les relations entre les entités ont été définies en respectant les règles de gestion du système académique :

- Une relation OneToOne existe entre Élève et DossierAdministratif, ce qui signifie qu'un élève possède un seul dossier administratif et qu'un dossier administratif est lié à un seul élève.
- Une relation ManyToOne relie Élève à Filière. Plusieurs élèves peuvent appartenir à une même filière, mais chaque élève est rattaché à une seule filière.
- Une relation OneToMany existe entre Filière et Élève, permettant à une filière de regrouper plusieurs élèves.
- Une relation ManyToMany est mise en place entre Cours et Filière, car une filière peut proposer plusieurs cours et un même cours peut être enseigné dans plusieurs filières.
- Une relation ManyToMany relie également Cours et Élève, puisqu'un élève peut suivre plusieurs cours et un cours peut être suivi par plusieurs élèves.

Ces relations ont été implémentées à l'aide des annotations JPA afin d'assurer une gestion correcte des associations en base de données.



Chapitre 1 : Modèle Conceptuel des Données (MCD)

Justification du modèle choisi

Le modèle conceptuel retenu est adapté aux besoins de l'application de gestion académique, car il respecte les règles métiers définies et permet une représentation fidèle du fonctionnement réel d'un établissement scolaire. La séparation claire des entités et l'utilisation de relations appropriées (OneToOne, OneToMany, ManyToMany) assurent une bonne cohérence des données et facilitent leur manipulation.

Ce modèle offre également une bonne évolutivité, permettant d'ajouter ultérieurement de nouvelles fonctionnalités ou entités sans remettre en cause la structure existante. Il constitue ainsi une base solide pour l'implémentation avec JPA et la base de données relationnelle.

Chapitre 2 : Architecture de l'Application

Présentation générale de l'architecture

L'application de gestion académique a été conçue selon une architecture en couches, permettant une organisation claire et structurée du code. Cette architecture facilite la compréhension du projet, améliore sa maintenabilité et permet une évolution progressive de l'application.

Le projet est organisé en plusieurs packages principaux : controller, service, repository et model, chacun ayant un rôle bien défini. Cette séparation permet de distinguer la gestion des requêtes, la logique métier, l'accès aux données et la représentation des entités du domaine académique.

La classe principale **GestionAcademiqueSpringbootApplication** assure le démarrage de l'application et l'initialisation du contexte Spring Boot.

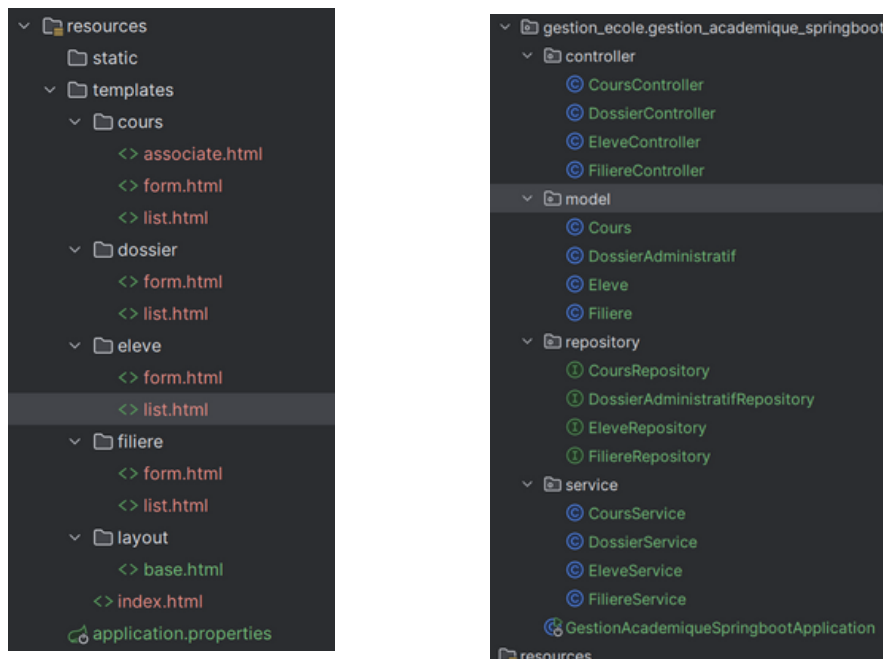
Architecture MVC (Model – View – Controller)

- Model :
- Le modèle correspond au package model. Il regroupe les entités JPA telles que Eleve, Filiere, Cours et DossierAdministratif. Ces classes représentent les données de l'application ainsi que leurs relations, et sont directement liées à la base de données via JPA.
- View :
- La vue est implémentée à l'aide de Thymeleaf, qui permet de créer des pages HTML dynamiques. Les vues affichent les données envoyées par les contrôleurs et permettent à l'utilisateur d'interagir avec l'application à travers des formulaires et des pages de consultation.
- Controller :
- Les contrôleurs, situés dans le package controller, tels que EleveController, FiliereController, CoursController et DossierController, reçoivent les requêtes HTTP provenant de l'utilisateur. Ils appellent les services correspondants, traitent les données et retournent les vues appropriées.

La couche service, bien qu'elle ne fasse pas directement partie du modèle MVC classique, joue un rôle central dans ce projet. Elle contient la logique métier et sert d'intermédiaire entre les contrôleurs et les repositories.

Chapitre 2 : Architecture de l'Application

Architecture MVC (Model – View – Controller)



Avantages de l'architecture MVC dans ce projet

L'utilisation de l'architecture MVC présente plusieurs avantages dans le cadre de ce projet de gestion académique. Elle permet avant tout une meilleure organisation du code, chaque couche ayant une responsabilité claire. Cette séparation rend le projet plus lisible et plus facile à maintenir.

De plus, l'architecture MVC facilite l'évolution de l'application. Par exemple, les interfaces graphiques peuvent être modifiées sans impacter la logique métier, et les règles de gestion peuvent évoluer sans affecter la structure des vues. Elle améliore également la réutilisabilité du code et simplifie les tests.

Chapitre 3 : Gestion des dépendances – Fichier pom.xml

La gestion des dépendances du projet est assurée à l'aide de Maven, à travers le fichier pom.xml, qui centralise l'ensemble des bibliothèques nécessaires au fonctionnement de l'application. Le projet est basé sur le parent Spring Boot, ce qui permet de bénéficier d'une configuration automatique et d'une compatibilité optimisée entre les différentes dépendances. Les dépendances Spring Boot Starter Web et Spring MVC sont utilisées pour le développement de l'application Web, tandis que Spring Data JPA assure la persistance des données et la gestion des entités. L'intégration de Thymeleaf permet la création des vues dynamiques, en cohérence avec l'architecture MVC adoptée. La connexion à la base de données est assurée via le driver MariaDB, et Lombok est utilisé pour réduire le code répétitif dans les classes du modèle. Enfin, des dépendances de test sont incluses afin de faciliter la validation de l'application. Ce choix de dépendances garantit une application stable, bien structurée et conforme aux bonnes pratiques du développement avec Spring Boot.

Chapitre 4 : Persistance des données avec JPA et MySQL

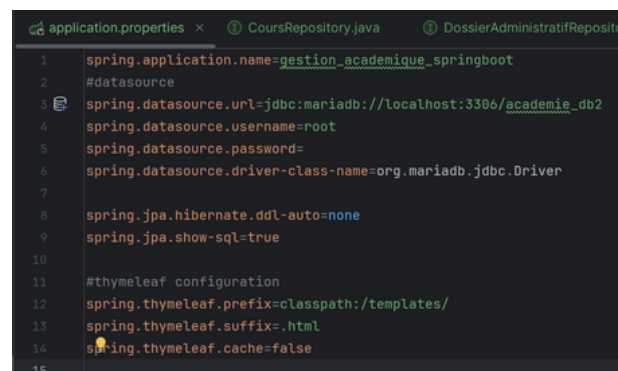
Présentation de JPA et de l'ORM

La persistance des données dans ce projet repose sur JPA (Java Persistence API), qui est une spécification permettant de gérer les données relationnelles sous forme d'objets Java. Grâce au principe de l'ORM (Object-Relational Mapping), les tables de la base de données sont représentées par des classes Java appelées entités. Cette approche permet de manipuler les données de manière orientée objet, sans écrire directement des requêtes SQL complexes. L'utilisation de JPA simplifie ainsi la gestion des données et améliore la lisibilité du code.

Configuration de la base de données MySQL

La base de données utilisée dans ce projet est MariaDB, configurée via le fichier `application.properties`. Les paramètres de connexion, tels que l'URL de la base de données, le nom d'utilisateur, le mot de passe et le driver JDBC, sont définis afin d'assurer la communication entre l'application Spring Boot et la base de données.

L'option `spring.jpa.show-sql=true` permet d'afficher les requêtes SQL générées par JPA, ce qui facilite le débogage. Le paramètre `spring.jpa.hibernate.ddl-auto=none` indique que la structure de la base de données est gérée manuellement, sans génération automatique des tables par Hibernate.



```
1 spring.application.name=gestion_academique_springboot
2 #datasource
3 spring.datasource.url=jdbc:mariadb://localhost:3306/academie_db2
4 spring.datasource.username=root
5 spring.datasource.password=
6 spring.datasource.driver-class-name=org.mariadb.jdbc.Driver
7
8 spring.jpa.hibernate.ddl-auto=none
9 spring.jpa.show-sql=true
10
11 #thymeleaf configuration
12 spring.thymeleaf.prefix=classpath:/templates/
13 spring.thymeleaf.suffix=.html
14 spring.thymeleaf.cache=false
15
```

Mapping des entités avec JPA

Les entités du projet sont mappées à la base de données à l'aide des annotations JPA telles que `@Entity`, `@Id` et `@GeneratedValue`. Chaque classe du package `model` correspond à une table dans la base de données.

Les repositories, qui héritent de `JpaRepository`, permettent d'effectuer automatiquement les opérations CRUD (création, lecture, mise à jour et suppression) sans implémentation manuelle. Cette approche simplifie l'accès aux données et réduit considérablement la quantité de code nécessaire.

Gestion des relations entre les entités

Les relations entre les entités sont gérées à l'aide des annotations JPA comme `@OneToOne`, `@OneToMany`, `@ManyToOne` et `@ManyToMany`. Afin d'optimiser les performances et d'éviter les problèmes de chargement paresseux, des requêtes personnalisées utilisant JPQL avec `JOIN FETCH` ont été mises en place dans certains repositories.

Par exemple, les repositories permettent de récupérer les cours avec leurs filières associées ou les dossiers administratifs avec les élèves et leurs filières en une seule requête. Cette stratégie améliore l'efficacité des accès aux données et garantit une meilleure cohérence lors de l'affichage des informations dans l'application

Chapitre 5 : Implémentation des opérations CRUD

CRUD de l'entité Élève

Les opérations CRUD de l'entité Élève permettent de gérer l'inscription et le suivi des élèves au sein de l'application. L'utilisateur peut ajouter un nouvel élève, consulter la liste des élèves, modifier ses informations ou le supprimer. Lors de la création ou de la modification d'un élève, celui-ci est obligatoirement associé à une filière existante. Les contrôleurs gèrent les requêtes HTTP, tandis que la logique métier est assurée par le service dédié, qui communique avec la base de données via le repository.

CRUD de l'entité Filière

La gestion des Filières repose sur des opérations CRUD simples permettant de créer, afficher, modifier et supprimer les filières académiques. Chaque filière est définie par un code, un nom et une description. Les filières servent de structure principale pour regrouper les élèves et organiser les cours. Les opérations sont implémentées à travers un contrôleur dédié, avec une séparation claire entre la couche présentation, la logique métier et l'accès aux données.

CRUD de l'entité Cours

Les opérations CRUD sur l'entité Cours permettent de gérer les enseignements proposés par l'établissement. En plus des opérations classiques, un cours peut être associé à une ou plusieurs filières. Cette association est gérée via des formulaires permettant de sélectionner les filières concernées. La logique mise en place garantit que les relations entre cours et filières sont correctement mises à jour avant l'enregistrement en base de données.

Gestion du Dossier Administratif

La gestion du Dossier Administratif permet de suivre les informations administratives liées à chaque élève. Chaque dossier est associé à un seul élève et contient notamment un numéro d'inscription et une date de création. Les opérations CRUD permettent de créer, modifier, consulter et supprimer les dossiers administratifs. Lors de la création d'un dossier, l'élève concerné est sélectionné afin d'assurer une liaison correcte entre les deux entités.

Règles métiers appliquées lors des opérations

Plusieurs règles métiers ont été appliquées afin d'assurer la cohérence des données. Un élève doit obligatoirement appartenir à une filière, un dossier administratif est lié à un seul élève, et les associations entre cours et filières sont gérées de manière contrôlée. De plus, certaines opérations utilisent des requêtes optimisées pour charger les relations nécessaires en une seule fois, ce qui améliore les performances et la fiabilité de l'application.

Chapitre 6 : Implémentation des vues avec Thymeleaf

Présentation de Thymeleaf

Thymeleaf est un moteur de templates côté serveur utilisé pour générer des pages HTML dynamiques dans les applications Spring Boot. Il permet d'intégrer directement des expressions dans les pages HTML afin d'afficher des données provenant du contrôleur. Dans ce projet, Thymeleaf est utilisé pour construire l'interface utilisateur tout en respectant l'architecture MVC, ce qui facilite la liaison entre la couche présentation et la logique métier.

Création des pages HTML dynamiques

Les pages de l'application sont organisées dans le dossier templates, avec une structure claire par entité (élève, filière, cours, dossier). Un layout principal (base.html) est utilisé pour centraliser la structure générale de l'interface, comme le menu de navigation, la barre latérale et l'en-tête.

Le contenu spécifique à chaque page est injecté dynamiquement à l'aide des mécanismes de fragments Thymeleaf, ce qui permet de réutiliser le même design pour toutes les pages et d'éviter la duplication du code HTML.

Interaction entre contrôleurs et vues

Les contrôleurs transmettent les données aux vues via l'objet Model. Ces données sont ensuite affichées dans les pages HTML à l'aide des expressions Thymeleaf (th:text, th:each, th:href, etc.).

Chaque action utilisateur (ajout, modification, suppression) est liée à une route définie dans les contrôleurs, ce qui assure une interaction fluide entre les vues et la logique métier. Cette approche garantit une navigation cohérente et une mise à jour dynamique des informations affichées.

Utilisation de Bootstrap pour l'interface utilisateur

Bootstrap est utilisé pour améliorer l'ergonomie et l'aspect visuel de l'application. Il permet de créer une interface moderne, responsive et adaptée à différents écrans. Les composants Bootstrap (cartes, boutons, tableaux, icônes) sont intégrés directement dans les pages Thymeleaf.

L'utilisation combinée de Bootstrap et de styles personnalisés permet d'obtenir une interface claire, structurée et facile à utiliser, tout en conservant une cohérence graphique sur l'ensemble de l'application.

Conclusion

Ce projet de gestion académique a permis de mettre en pratique les concepts fondamentaux du développement d'applications Web avec Spring Boot. À travers la mise en place de l'architecture MVC, la modélisation des données avec JPA, l'implémentation des opérations CRUD et la création des interfaces avec Thymeleaf et Bootstrap, une application fonctionnelle et bien structurée a été développée.

Ce travail m'a permis de renforcer ma compréhension de l'organisation d'un projet Spring, de la gestion des relations entre entités et de l'interaction entre la couche backend et la couche frontend. Il constitue une base solide pour le développement d'applications Web plus complètes et évolutives.

AcadémixPro

Tableau de bord

Gestion des Elèves

Filières

Cours

Dossiers

Info

Bienvenue, Administrateur

Gestion des Cours

Organisez vos matières académiques

Nouveau cours

Code	Intitulé	Filières	Actions
JAVA	Programmation Java	Informatique	
WEB	Développement Web	Informatique Génie Industriel	
BD	Bases de Données	Informatique	
ELC	Électronique de base	génie environnement	

AcadémixPro

Tableau de bord

Gestion des Elèves

Filières

Cours

Dossiers

Info

Bienvenue, Administrateur

Gestion des Elèves

Liste complète des étudiants inscrits

Nouvel élève

Rechercher un élève...Trier par: Nom

Matricule	Nom complet	Filière	Statut	Actions
E001	Dupont1 Jean Pas d'email	Informatique	Actif	
E002	Martin Sara saramartin@email.com	Informatique	Actif	
E003	Amrani Youssef y.amrani@email.com	Génie Electrique	Actif	
E004	Benali Khadija k.benali@email.com	Génie Civil	Actif	
AMRANI	Iayla Iayla Pas d'email	Génie Electrique	Actif	

AcadémixPro

Tableau de bord

Gestion des Elèves

Filières

Cours

Dossiers

Info

Bienvenue, Administrateur

Gestion des Filières

Organisez vos parcours académiques

Nouvelle filière

INO

Informatique

Filière Informatique

Élèves 25Cours 8

Gérer

GE

Génie Électrique

Filière Génie Électrique

Élèves 25Cours 8

Gérer

GC

Génie Civil

Filière Génie Civil

Élèves 25Cours 8

Gérer

GI

Génie Industriel

Filière Génie Industriel

Élèves 25Cours 8

Gérer

GEN

génie environnement

génie environnement

Élèves 25Cours 8

Gérer

AcadémixPro

Tableau de bord

Gestion des Elèves

Filières

Cours

Dossiers

Info

Bienvenue, Administrateur

Gestion des Dossiers

Liste complète des dossiers administratifs

Nouveau dossier

Rechercher un dossier...Trier par: Date de création

Numéro	Date création	Élève	Filière	Actions
DOSS-2024-001	01/09/2024	J Dupont1 Jean E001	Informatique	
DOSS-2024-002	02/09/2024	S Martin Sara E002	Informatique	
DOSS-2024-003	03/09/2024	Y Amrani Youssef E003	Génie Electrique	
DOSS-2024-004	05/09/2024	K Benali Khadija E004	Génie Civil	

Total: 4 dossier(s)Précédent123Suivant